

Experiment-5

AIM- To interface the LPS22HB sensor via I2C and obtain pressure data from environmental monitoring.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock Selection:** Go to RCC and configure clock as “By pass Clock Source” as shown below:

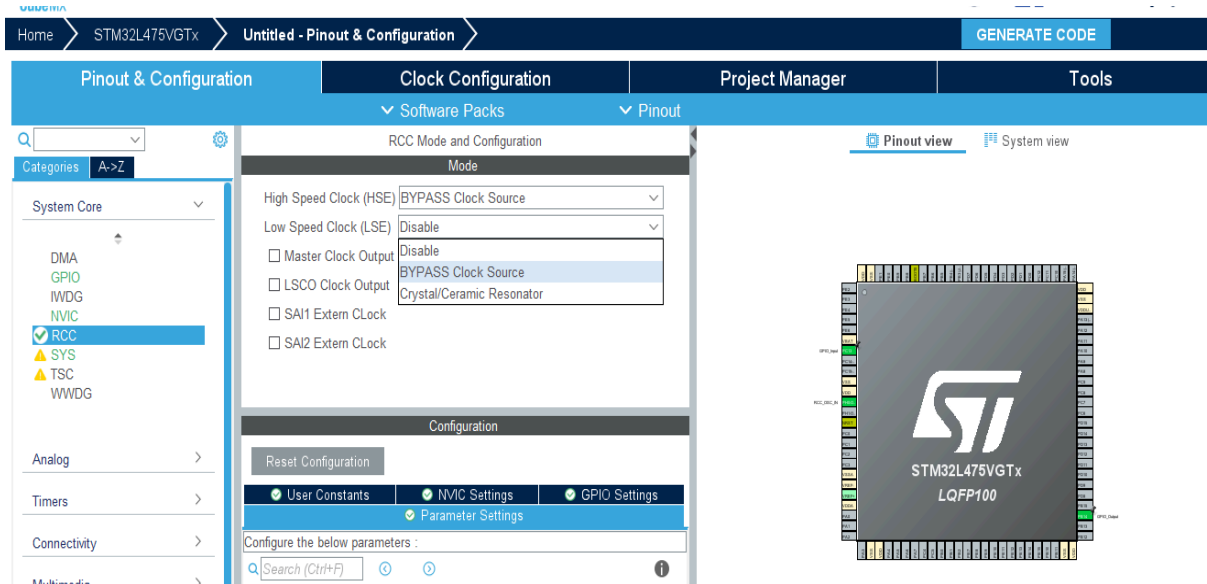


Fig-5.1 – Clock Selection

4. As per user manual of board ST LINK is internally connected to PB6(Tx) and PB7(Rx) of USART1.

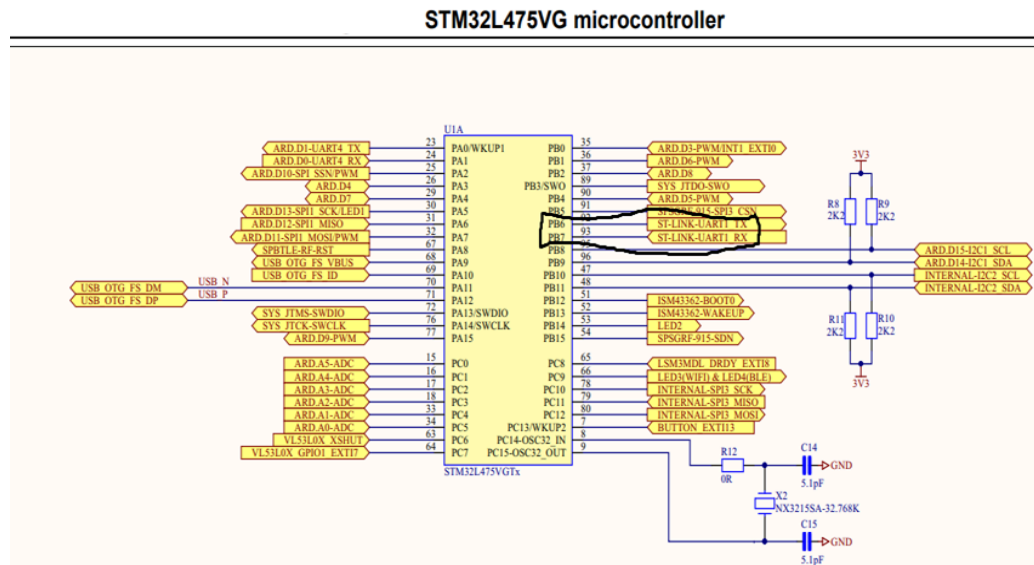


Fig-5.2 – STM32 Board Pin Configuration

- Under “connectivity” click on “USART1” and make PB6 as USART1_Tx and PB7 as USART2_Rx as shown below:

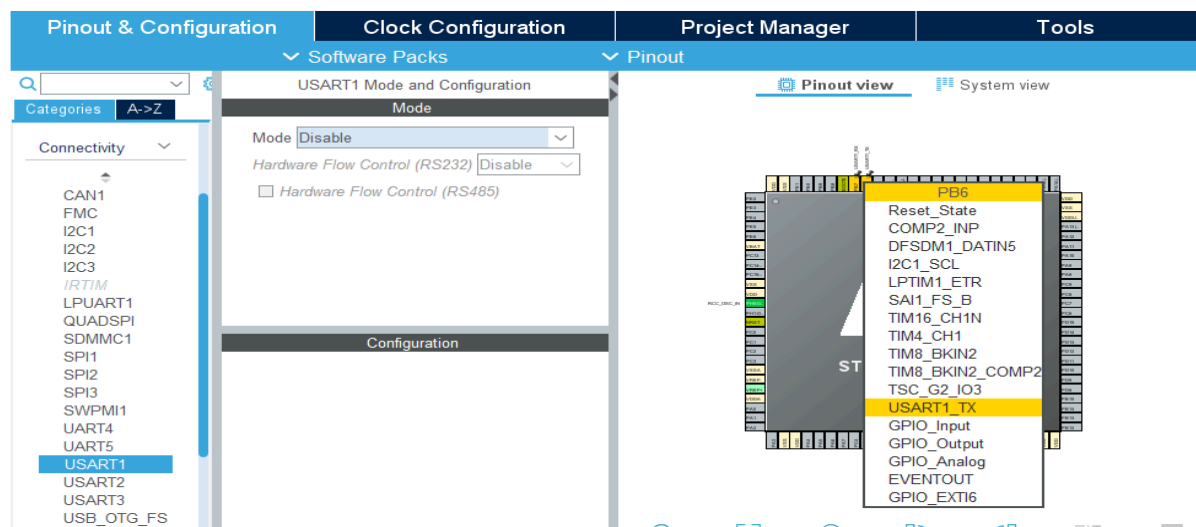


Fig-5.3 – STM32 Board Pinout and Configuration

- Select USART1 and click on “Asynchronous” in “Mode” as shown below :

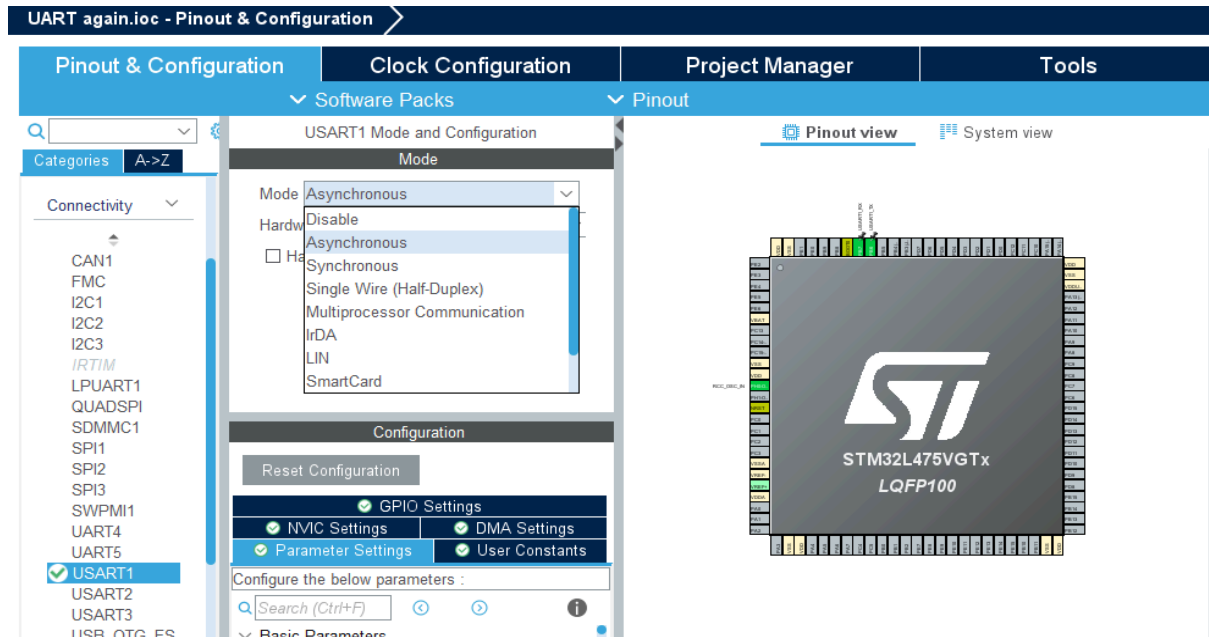


Fig-5.4 – Selection of Asynchronous mode

7. Go to “connectivity” and also select I2C1 in mode.
8. Click on PB8 and select I2C1_SCL to make PB8 pin as clock source pin as shown below:

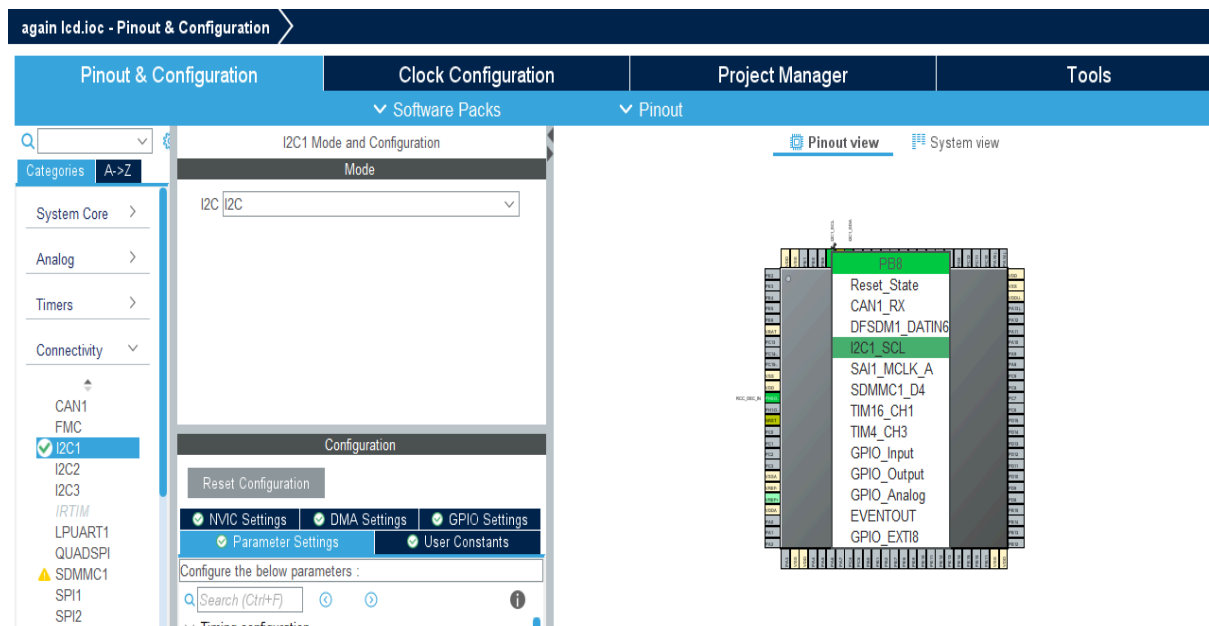
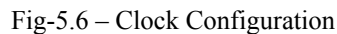


Fig-5.5 – Settings of Basic Parameters

9. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.



-
- The screenshot shows a Windows File Explorer window. The address bar at the top indicates the path: This PC > Local Disk (C:) > Users > Ienovo > STM32Cube > Repository > STM32Cube_FW_L4_V1.17.2 > Drivers. The main pane displays a list of folders: BSP, CMSIS, and STM32L4xx_HAL_Driver. The 'BSP' folder is selected. The right-hand pane shows the context menu for the selected folder, with 'Copy' highlighted. The context menu includes options like 'Open', 'Open in new window', 'Pin to Quick access', 'Add to VLC media player's Playlist', 'Play with VLC media player', 'Scan with Microsoft Defender...', 'Give access to', 'Open "Drivers" in WinZip', 'WinZip PDF Express', 'WinZip SafeShare', 'WinZip', 'Shared Folder Synchronization', 'Restore previous versions', 'Include in library', 'Pin to Start', 'Add to archive...', 'Add to "BSP.rar"', 'Compress and email...', 'Compress to "BSP.rar" and email', 'Send to', 'Cut', and 'Copy'.

16. Paste the copied BSP drivers in the drive folder as shown below:

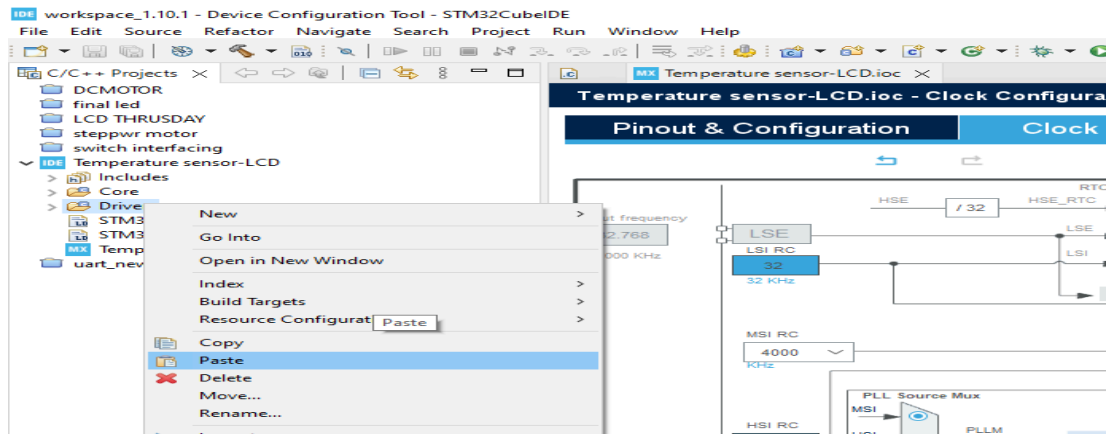


Fig-5.8 – Paste BSP drivers in project folder

17. Delete all folders in BSP, except B-L475E-IOT01 and the component as shown:

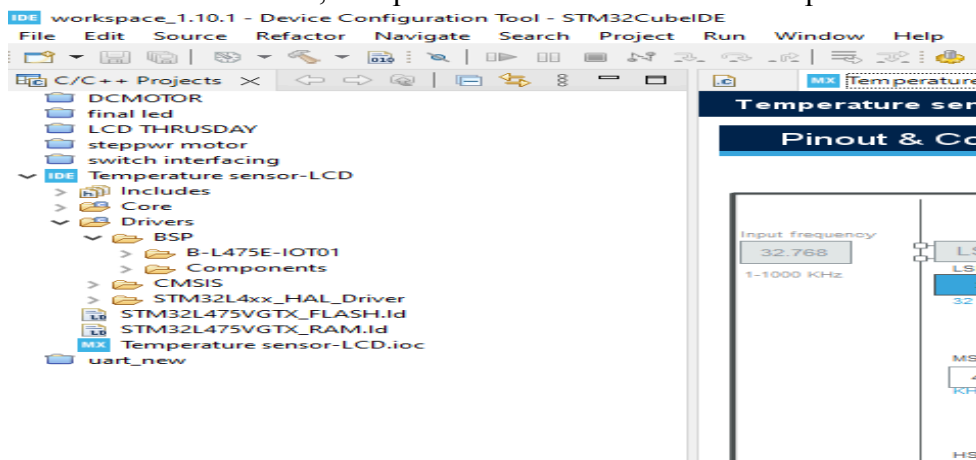


Fig-5.9 – B-L475E-IOT01 and component folder in BSP drivers

18. Keep only these folders in B-L475E-IOT01 and the component as shown in figure:

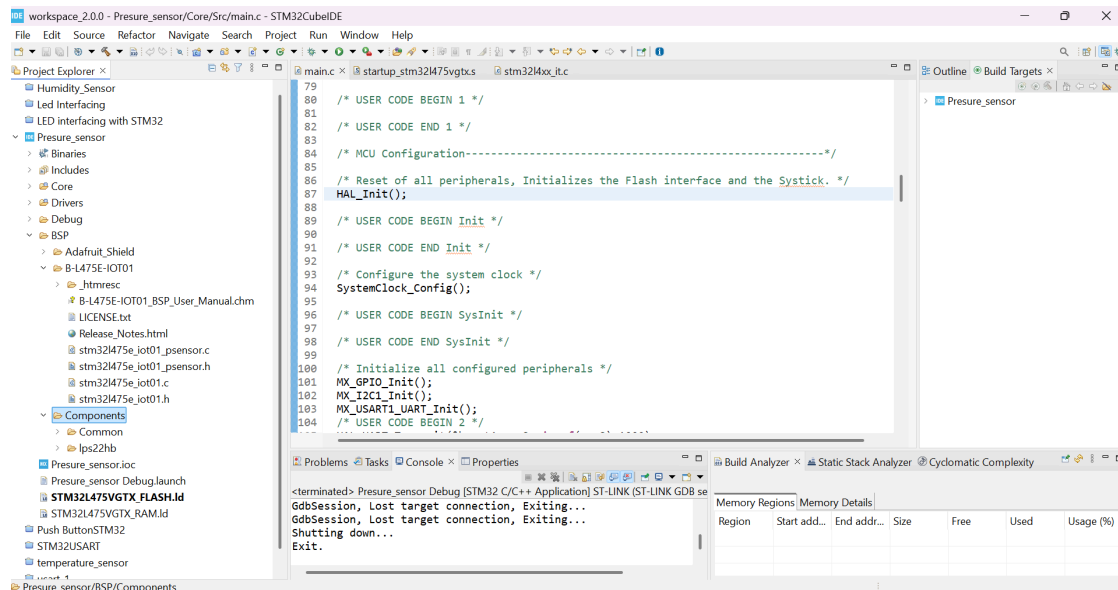


Fig-5.10 – Folders in B-L475E-IOT01 and Component

19. Update include paths: From Project menu or File menu, go to Properties > C/C++ Build > Settings > Tool Settings > MCU GCC Compiler > Include paths and click “add” icon.png to include the new paths as shown below:

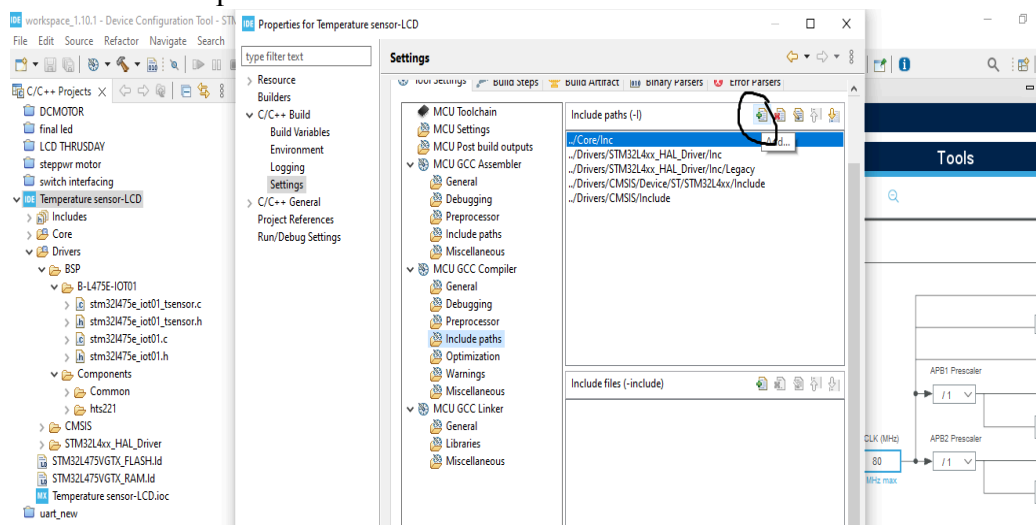


Fig-5.11 – Path settings

20. Add ../Drivers/BSP/B-L475E-IOT01 and ../Drivers/BSP/Components/hts221 paths as addresses given below :
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\B-L475E-IOT01“ and
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\Components\hts221

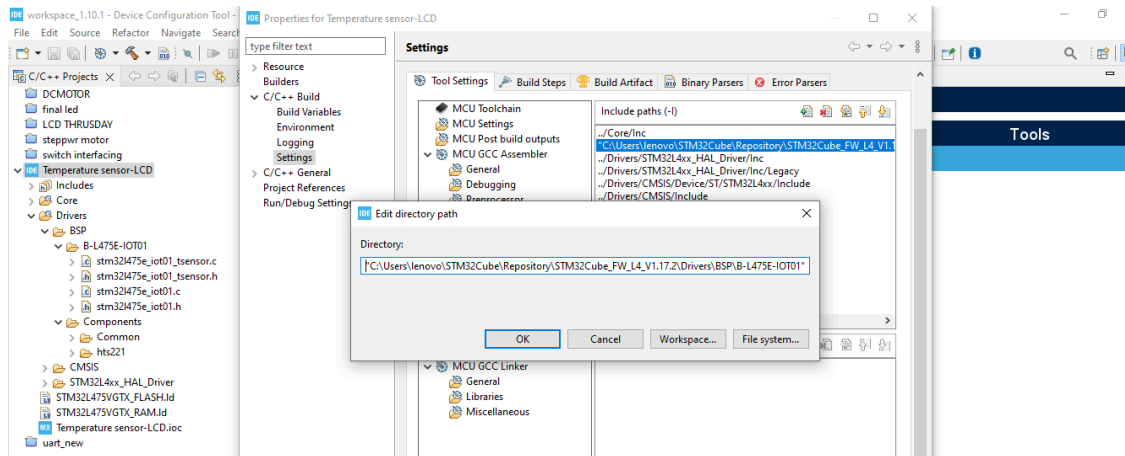


Fig-5.12 – Folders Path settings

21. After adding both path click “apply and close” as shown below:

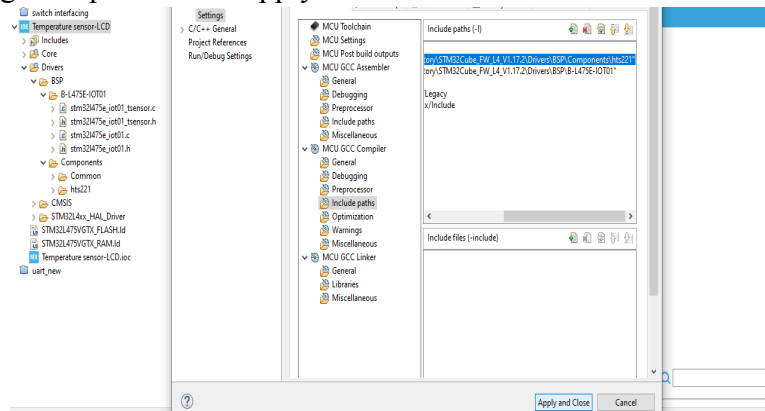


Fig-5.13 – Apply and close setting

22. Go to properties of project>go to settings>go to MCU/MPU settings>Click to float as >apply and close as shown in figure:

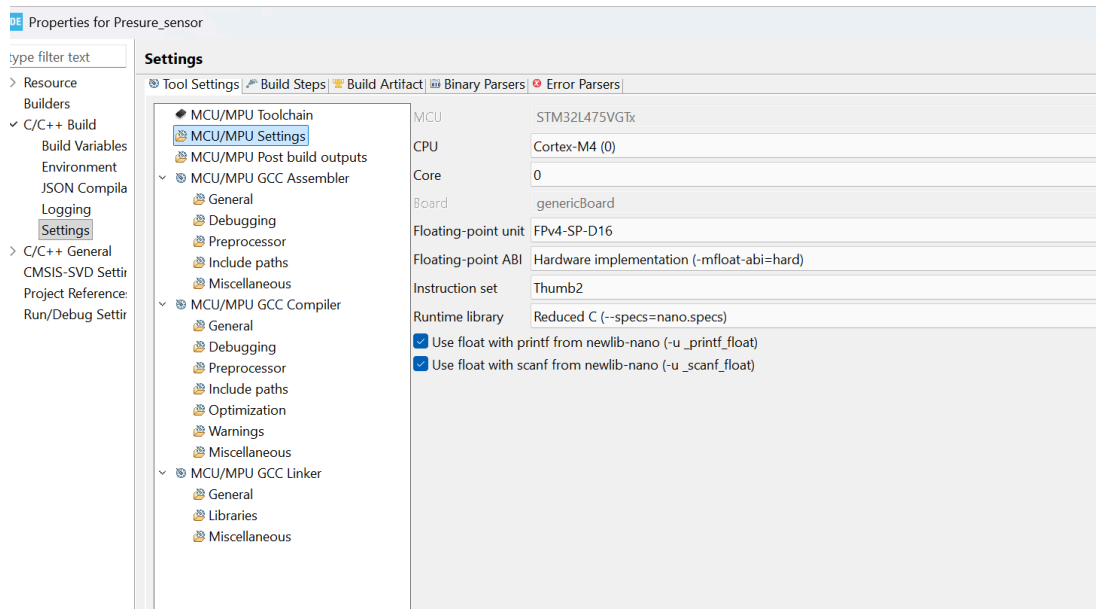


Fig-5.14 – MCU/MPU setting

23. Copy lps22hb.c, stm32l475e_iot1_psensor.c, stm32l475e_iot1.c in src folder and lps22hb.c in Inc from BSP folder.
24. Then “Build” and go to main.c file.
25. Include the header files in main.c files.

```
/* Includes */
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_psensor.h"
#include "stdlib.h"
#include <math.h>
#include <string.h>
#include <stdio.h>
```

26. Write the following instructions:

```
/* USER CODE BEGIN PV */
char str[40];
float p_value = 0; // Measured pressure value
char str_tmp[100] = ""; // Formatted message to display the pressure value
uint8_t msg3[] = "=====> Pressure sensor lps22hb initialized \r\n ";
/* USER CODE END PV */
```

27. Write the following instructions as given:

```
/* USER CODE BEGIN 2 */
HAL_UART_Transmit(&huart1,msg3,sizeof(msg3),1000);
if (BSP_PSENSOR_Init() != PSENSOR_OK)
{
    // Optional: Send an error message if init fails
    char err_msg[] = "Sensor Init Error\r\n";
```



```
    HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), 1000);  
}  
/* USER CODE END 2 */
```

28. Write the following instructions in while (1) loop.
p_value = BSP_PSENSOR_ReadPressure();
sprintf(str_tmp, "%.1f\n\r", p_value);
HAL_UART_Transmit(&huart1, (uint8_t*)str_tmp, **strlen**(str_tmp), 1000); // Use strlen()
HAL_Delay(1000);
29. Now connect the board and click on “Build”, “debug”.
30. Click on “Switch” on popup and click on “Resume”, the program will start execute.
31. Now, open the console “Real term” and set the baud rate as used in program (in this program it is 115200), select port (as per use) and click on “change” on the real term console.

OBSERVATION: The output value of pressure will display on real term as shown below:

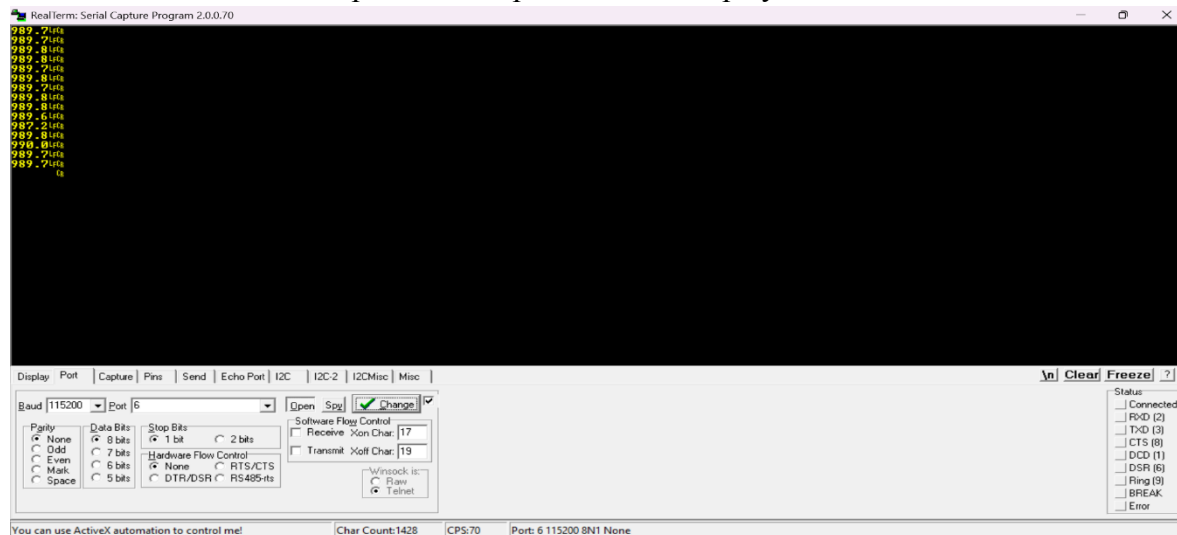


Fig: 5.15 – Output from experiment

Reflection Summary

Q1. How has this experiment improved your understanding of interfacing an I2C-based pressure sensor (LPS22HB) with the STM32 microcontroller and using BSP drivers in an embedded project?

This experiment helped me understand how an I2C sensor like the LPS22HB is actually connected and used with an STM32 board in a real project. I learned how BSP drivers make sensor interfacing easier because they already provide ready-to-use functions for initialization and reading data. Instead of writing everything from scratch, I could focus more on configuration and application logic. Overall, it gave me a clear idea of how software libraries and hardware work together in embedded systems.

Q2. What challenges did you encounter while configuring peripherals (clock, USART1, I2C1) and setting include paths, and how did you resolve them during the experiment?

The main challenges were setting the correct clock configuration and making sure USART1 and I2C1 were configured properly in CubeMX. At first, the project showed errors because the BSP and component include paths were not added correctly. This was solved by carefully adding the required folders in the include path settings and rebuilding the project. After that, the code compiled properly and the sensor data started showing on RealTerm.

Q3. In what ways can the pressure data acquired and displayed on RealTerm be applied in real-world environmental monitoring or IoT applications?

The pressure data can be used in many real-world applications like weather monitoring, altitude measurement, and environmental observation systems. For example, it can help in predicting weather changes or monitoring atmospheric conditions in smart cities. In IoT systems, this data can be sent to cloud platforms for logging and analysis. This makes the system useful for remote monitoring and decision-making applications.